

UNIVERSIDADE DE SÃO PAULO

Instituto de Ciências Matemáticas e de Computação

Cloud-Project - Plataforma de monitoramento ambiental
integrada usando MQTT, Kafka e Mongo DB

*Beatriz Vieira da Silva Andrade, Eduardo Garcia de Gaspari
Valdeão, Gabriel Vasconcelos de Arruda*



São Carlos – SP

Cloud-Project - Plataforma de monitoramento ambiental integrada
usando MQTT, Kafka e Mongo DB

**Beatriz Vieira da Silva Andrade, Eduardo Garcia de Gaspari Valdejão,
Gabriel Vasconcelos de Arruda**

***Orientador:* Prof. Dr. Júlio Cezar Estrella**

Projeto final apresentado a disciplina SSC0158,
Instituto de Ciências Matemáticas e de Computação –
ICMC-USP, como requisito parcial para obtenção de
aprovação na disciplina.

Área de Concentração: Computação em Nuvem e
Arquitetura Orientadas a Serviços

**USP – São Carlos
Junho de 2025**

RESUMO

VIEIRA, BEATRIZ. VALDEJÃO, EDUARDO. ARRUDA, GABRIEL. ***Cloud-Project - Plataforma de monitoramento ambiental integrada usando MQTT, Kafka e Mongo DB***. 2025. 15 f. Projeto (Graduação) – Instituto de Ciências Matemáticas e de Computação (ICMC/USP), São Carlos – SP.

O monitoramento ambiental remoto tem se tornado cada vez mais relevante em ambientes acadêmicos, industriais e residenciais, especialmente pela necessidade de acompanhar parâmetros como temperatura e umidade de forma contínua e confiável. Nesse contexto, soluções baseadas em *Internet das Coisas (IoT)* tornam-se essenciais, permitindo a coleta de dados em tempo real e o controle eficiente do ambiente monitorado.

Este trabalho propõe o desenvolvimento de uma solução completa de monitoramento ambiental, utilizando um microcontrolador ESP32 conectado a sensores, com transmissão dos dados via protocolo MQTT, roteamento por meio do Apache Kafka e armazenamento no MongoDB. Para complementar, foi desenvolvida uma aplicação web capaz de exibir os dados coletados em gráficos, facilitando o acompanhamento e a análise das condições do ambiente. A integração de todos os serviços foi realizada utilizando Docker e Docker Compose, garantindo escalabilidade, replicabilidade e facilidade no deployment da solução em ambientes na nuvem.

Palavras-chave: IoT, Sensores, Nuvem.

SUMÁRIO

1	INTRODUÇÃO	4
1.1	Contextualização	4
1.2	Descrição do Problema e Motivação	4
1.3	Objetivos	5
2	METODOLOGIA DE DESENVOLVIMENTO	6
2.1	Visão Geral da Solução	6
2.2	Componentes de Hardware	6
2.3	Componentes de Software	7
2.4	Fluxo de Dados da Solução	7
2.5	Execução da Solução	8
2.6	Desenvolvimento da Aplicação Web	8
2.7	Ambiente de Deploy com Docker	9
3	RESULTADOS	11
3.1	Coleta e Armazenamento dos Dados	11
3.2	Análise e Visualização dos Dados Coletados na Aplicação Web	11
3.3	Desafios Técnicos e Limitações	12
3.4	Validação da Solução	13
3.5	Pendências do Projeto	13
3.6	Discussão dos Resultados	13
4	CONCLUSÃO	14
4.1	Considerações Finais	14
4.2	Contribuições	14
4.3	Desafios e Limitações	14
4.4	Trabalhos Futuros	15
4.5	Encerramento	15
4.6	Referências	15

INTRODUÇÃO

1.1 Contextualização

O avanço das tecnologias de sensoriamento, conectividade e computação tem impulsionado significativamente a *Internet das Coisas (IoT)*, permitindo que dispositivos físicos colem, transmitam e processem dados em tempo real. Aplicações de IoT no monitoramento ambiental são cada vez mais relevantes, utilizando sensores de temperatura, umidade, qualidade do ar e outros parâmetros em ambientes industriais, agrícolas, comerciais e residenciais para promover eficiência, segurança e conforto.

Com o crescimento no número de dispositivos, surgem desafios relacionados ao processamento e armazenamento de grandes volumes de dados. A computação em nuvem surge como solução eficiente, oferecendo escalabilidade, alta disponibilidade, elasticidade e redução de custos operacionais, além de permitir acesso remoto aos dados.

1.2 Descrição do Problema e Motivação

Embora existam soluções comerciais de monitoramento ambiental, muitas apresentam limitações quanto a custo, escalabilidade e flexibilidade. Frequentemente são dependentes de plataformas fechadas, com pouca possibilidade de customização e altos custos para expansão em ambientes com múltiplos sensores.

Soluções tradicionais, focadas em armazenamento local, não oferecem escalabilidade, resiliência ou acesso remoto eficiente. Isso torna necessário desenvolver uma arquitetura capaz de ser:

- **Distribuída**, permitindo sensores em diferentes locais;
- **Escalável**, suportando crescimento no volume de dados e número de dispositivos;
- **Flexível**, facilitando integração, customização e visualização;
- **De baixo custo**, tanto em hardware quanto em software.

Este projeto responde a essa demanda, ao mesmo tempo em que consolida conceitos teóricos da disciplina de Computação em Nuvem, como comunicação entre dispositivos, mensageria assíncrona, processamento distribuído e containerização. Além de atender às necessidades acadêmicas, a proposta é aplicável a cenários reais, utilizando tecnologias amplamente adotadas no mercado, como MQTT, Apache Kafka, MongoDB, Flask e Docker.

1.3 Objetivos

O objetivo principal é desenvolver uma plataforma de monitoramento ambiental baseada em IoT e computação em nuvem, capaz de realizar a coleta, transmissão, processamento, armazenamento e visualização dos dados de temperatura e umidade de forma eficiente, escalável e contínua.

São objetivos específicos:

- **Projetar e implementar um sistema de sensoriamento** com ESP32 e sensor DHT22.
- **Realizar comunicação dos sensores** via protocolo MQTT.
- **Roteamento dos dados** utilizando Apache Kafka para garantir desacoplamento dos serviços e escalabilidade.
- **Persistir os dados** em um banco NoSQL MongoDB, estruturando-os para consultas e análises.
- **Desenvolver uma aplicação web**, utilizando Flask, para visualização dos dados em tempo real.
- **Containerizar toda a infraestrutura** com Docker e Docker Compose, garantindo portabilidade, replicação e facilidade de implantação na nuvem.

A solução proposta é replicável e adaptável, podendo ser aplicada tanto em ambientes acadêmicos quanto no mercado, atendendo a diferentes demandas de monitoramento e análise de dados ambientais.

METODOLOGIA DE DESENVOLVIMENTO

2.1 Visão Geral da Solução

O desenvolvimento foi dividido nas seguintes etapas:

- Projeto e configuração do hardware responsável pela coleta dos dados.
- Implementação da comunicação e processamento dos dados utilizando MQTT, Apache Kafka e scripts intermediários em Python.
- Armazenamento dos dados no MongoDB.
- Desenvolvimento de uma aplicação web em Flask para visualização dos dados em tempo real.
- Containerização de todos os serviços utilizando Docker e orquestração com Docker Compose, permitindo o deploy completo em uma máquina virtual na nuvem.

2.2 Componentes de Hardware

O hardware utilizado no desenvolvimento da solução é composto por um microcontrolador ESP32, responsável pela leitura dos dados ambientais, conectado a um sensor DHT22, que mede temperatura e umidade.

Tabela 1 – Componentes de hardware utilizados.

Componente	Especificação	Função
ESP32-WROOM-32	Microcontrolador	Coleta dados e publica no MQTT
DHT22	Sensor de temperatura e umidade	Medição dos dados ambientais
Protoboard e jumpers	Padrão	Conexões físicas
Fonte USB 5V	Fonte de alimentação	Alimentação do ESP32

As conexões físicas entre o ESP32 e o sensor DHT22 foram realizadas conforme a Tabela 2.

Tabela 2 – Conexões entre ESP32 e DHT22.

Pino ESP32	Pino DHT22
3.3V	VCC
GND	GND
GPIO4	DATA

2.3 Componentes de Software

O sistema desenvolvido integra os seguintes componentes de software:

- **Arduino IDE:** utilizado para desenvolvimento e upload do código no ESP32.
- **Mosquitto MQTT Broker:** responsável pela recepção das mensagens publicadas pelo ESP32.
- **Apache Kafka e Zookeeper:** plataforma de mensageria e processamento distribuído de dados em tempo real.
- **MongoDB:** banco de dados NoSQL utilizado para armazenamento dos dados coletados.
- **Python 3.8+:** desenvolvimento dos scripts intermediários para integração MQTT → Kafka e Kafka → MongoDB.
- **Flask:** desenvolvimento de uma aplicação web para visualização dos dados armazenados, com geração de gráficos dinâmicos.
- **Docker e Docker Compose:** containerização e orquestração de todos os serviços do sistema, possibilitando execução padronizada e escalável em uma máquina virtual na nuvem.

2.4 Fluxo de Dados da Solução

O funcionamento do sistema segue o seguinte fluxo:

1. O ESP32 lê os dados de temperatura e umidade por meio do sensor DHT22.
2. Os dados coletados são publicados no tópico MQTT `sensor_temperatura` via broker Mosquitto.
3. Um script Python consome as mensagens do MQTT e as publica no tópico `sensor_temperatura` do Apache Kafka.
4. Um segundo script Python consome as mensagens do Kafka e realiza a persistência dos dados no MongoDB, na coleção `dadosTempTopico`.

5. A aplicação web desenvolvida em Flask realiza consultas periódicas no MongoDB e apresenta os dados coletados em gráficos interativos, acessíveis via navegador.

A Figura 1 apresenta o diagrama representando o fluxo de dados da solução.

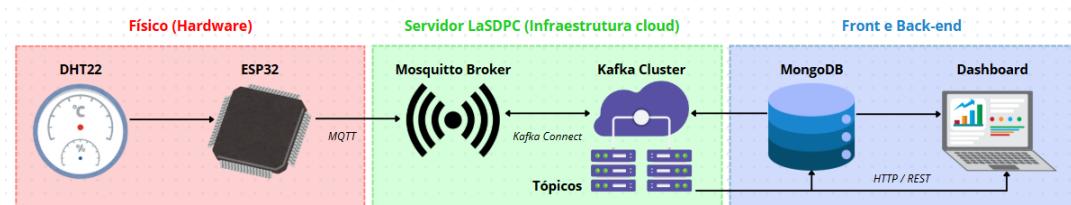


Figura 1 – Fluxo de dados da solução desenvolvida.

2.5 Execução da Solução

O processo de execução da solução ocorre conforme os seguintes passos:

1. Subir o código `configesp32.ino` no ESP32 utilizando a Arduino IDE, configurando os parâmetros da rede Wi-Fi e o endereço do broker MQTT.
2. Na máquina virtual, executar o comando `docker-compose up -d`, que inicia todos os serviços (Mosquitto, Kafka, Zookeeper, MongoDB, scripts Python e Flask).
3. Verificar a coleta de dados através dos logs do broker MQTT, do Kafka e do MongoDB.
4. Acessar o painel web desenvolvido com Flask via navegador, utilizando o IP da máquina virtual, para visualizar os dados em tempo real na forma de gráficos.
5. Realizar consultas e análises diretamente no MongoDB, se desejado, utilizando ferramentas como MongoDB Compass ou mongosh.

2.6 Desenvolvimento da Aplicação Web

Como parte da solução desenvolvida, foi implementada uma aplicação web utilizando o microframework Flask, com o objetivo de disponibilizar os dados coletados de forma visual, acessível e em tempo real.

A aplicação conecta-se diretamente ao banco de dados MongoDB, realiza consultas na coleção `dadosTempTopico` e apresenta os dados em forma de gráficos. Essa visualização permite que os usuários acompanhem, de maneira intuitiva, a evolução dos dados de temperatura e umidade do ambiente monitorado.

O backend da aplicação, desenvolvido no arquivo `app.py`, é responsável por definir a rota principal (`/`), que executa uma consulta no MongoDB, ordenando os registros e extraindo

os valores de temperatura, umidade e timestamps. Esses dados são enviados para o template `index.html`, que faz uso de bibliotecas gráficas (como `Chart.js`) para renderizar os gráficos dinâmicos no navegador.

O painel é acessível remotamente por meio do IP da máquina virtual onde o ambiente está executando, na porta exposta pelo container da aplicação. Com isso, a solução oferece não apenas o armazenamento dos dados, mas também uma camada de apresentação visual, fundamental para monitoramento em tempo real e análise histórica.

A Figura 2 ilustra um exemplo da interface desenvolvida.

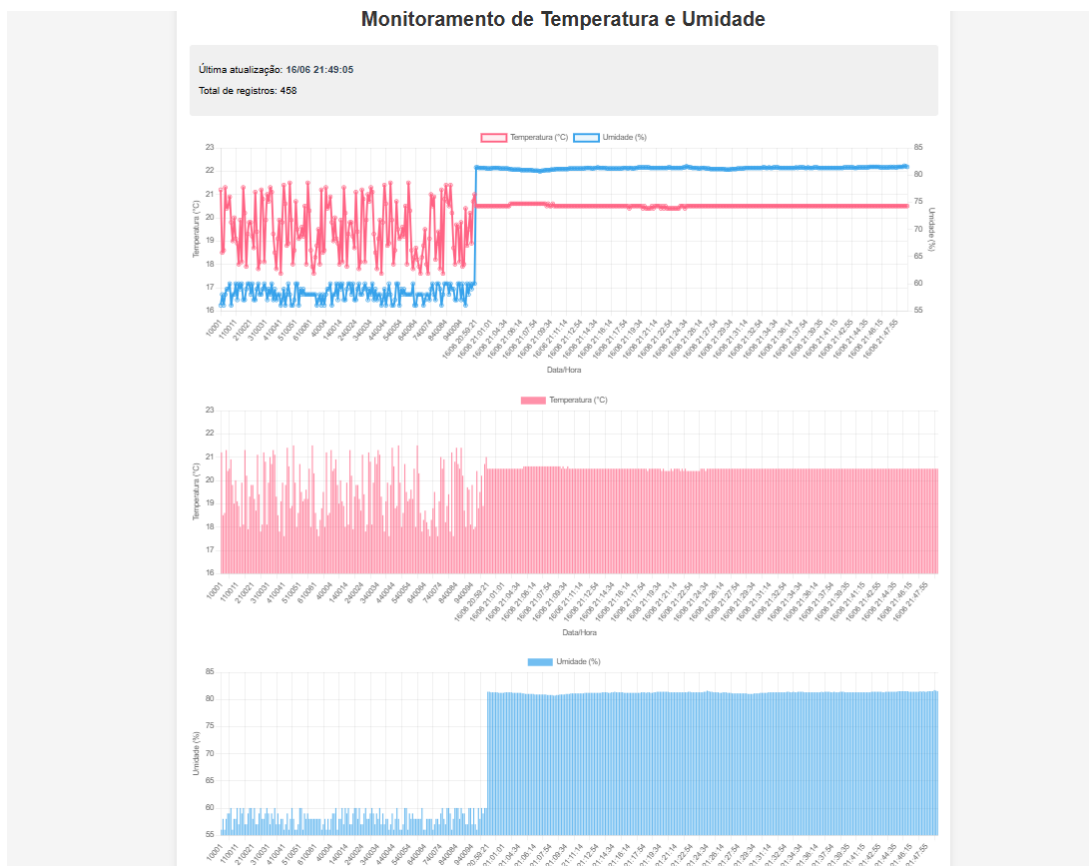


Figura 2 – Interface web desenvolvida com Flask para visualização dos dados.

2.7 Ambiente de Deploy com Docker

Para simplificar o deployment e garantir a replicação consistente do ambiente, todos os serviços foram containerizados utilizando Docker, e orquestrados com Docker Compose. Essa abordagem permite que toda a solução — incluindo comunicação, processamento, armazenamento e visualização dos dados — funcione de forma integrada em qualquer ambiente que suporte Docker.

A Tabela 3 apresenta os serviços definidos no arquivo `docker-compose.yml` e seus respectivos papéis na solução.

Tabela 3 – Serviços definidos no Docker Compose.

Serviço	Função
Mosquitto	Broker MQTT responsável pela recepção dos dados do ESP32.
Kafka	Sistema de mensageria e processamento de fluxo de dados.
Zookeeper	Serviço auxiliar para gerenciamento e coordenação dos brokers Kafka.
MongoDB	Banco de dados NoSQL para persistência dos dados coletados.
projeto_front (Flask)	Aplicação web para visualização dos dados em gráficos.
mqtt_kafka	Script Python responsável pela integração dos dados do broker MQTT para o Kafka.
kafka_mongo	Script Python responsável pela integração dos dados do Kafka para o MongoDB.

Os serviços se comunicam internamente por meio de uma rede virtual definida no Compose, garantindo isolamento do ambiente e facilitando a gestão das conexões entre os containers. O comando `docker-compose up -d` é suficiente para inicializar todos os serviços automaticamente, permitindo que o sistema opere de forma estável e escalável na máquina virtual utilizada.

Dessa forma, a arquitetura baseada em containers garante que toda a solução seja facilmente replicável, portátil e escalável, permitindo o deployment consistente tanto em ambientes locais quanto na nuvem.

RESULTADOS

3.1 Coleta e Armazenamento dos Dados

A partir da implementação da solução, foi possível realizar a coleta contínua de dados de temperatura e umidade utilizando o sensor DHT22 conectado ao microcontrolador ESP32. Esses dados foram publicados via protocolo MQTT, processados pelo Apache Kafka e, posteriormente, armazenados no banco de dados MongoDB.

A Tabela 4 apresenta uma amostra dos dados coletados e registrados na coleção `dadosTempTopico` do MongoDB.

Tabela 4 – Amostra dos dados armazenados no MongoDB.

ID	Tempo (Data/Hora)	Temperatura (°C)	Umidade (%)
1	16/06 21:00:11	20.6	81.3
2	16/06 21:05:04	20.6	81.1
3	16/06 21:07:04	20.6	80.9
4	16/06 21:09:44	20.6	81.0
5	16/06 21:13:24	20.5	81.3
6	16/06 21:16:24	20.5	81.2
7	16/06 21:21:04	20.5	81.3
8	16/06 21:24:24	20.4	81.4
9	16/06 21:49:05	20.5	81.5

3.2 Análise e Visualização dos Dados Coletados na Aplicação Web

Além da persistência dos dados no MongoDB, foi desenvolvido um painel web utilizando Flask, permitindo a visualização dos dados de forma intuitiva e acessível.

Essa aplicação conecta-se ao MongoDB e gera gráficos dinâmicos que apresentam, em tempo real, os valores de temperatura e umidade. A Figura 3 ilustra a interface do painel desenvolvido.

A análise dos dados coletados permite observar a evolução das variáveis de temperatura e umidade ao longo do tempo. Os gráficos demonstram que o sistema foi capaz de realizar leituras

consistentes ao longo do tempo, com os dados sendo coletados, transmitidos e armazenados sem interrupções, validando a robustez da solução proposta. Na figura observamos registros iniciais aleatórios (de teste), e registros reais após conectarmos o sensor.

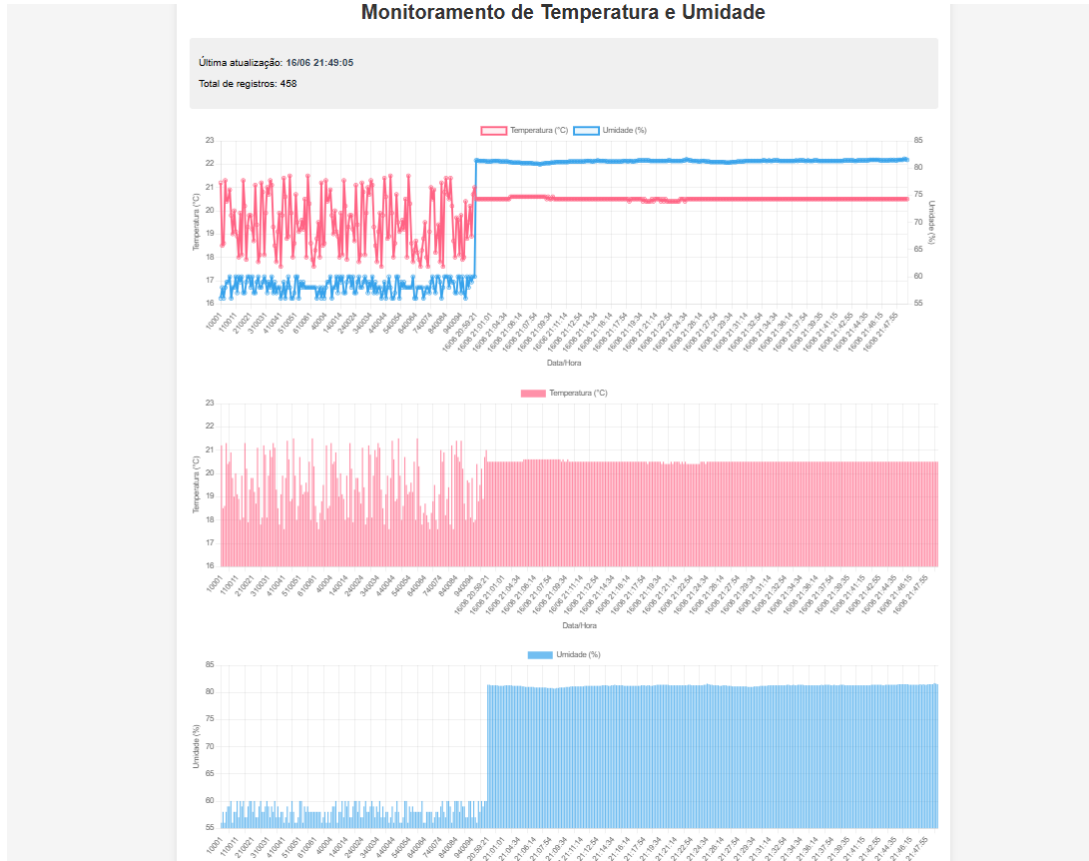


Figura 3 – Interface da aplicação web Flask exibindo os dados coletados.

A funcionalidade do painel demonstra que a solução proposta não apenas realiza a coleta, transmissão e armazenamento dos dados, mas também oferece uma interface de visualização eficiente, facilitando o monitoramento e a análise dos dados em tempo real.

3.3 Desafios Técnicos e Limitações

Durante o desenvolvimento e execução do projeto, foram enfrentados alguns desafios, tais como:

- **Configuração do Apache Kafka:** exigiu ajustes específicos no ambiente, além do gerenciamento do Zookeeper para funcionamento correto.
- **Estabilidade da conexão Wi-Fi:** em alguns momentos houve perda de pacotes entre o ESP32 e o broker MQTT devido à instabilidade da rede.

- **Integração entre serviços:** ajustes foram necessários nos scripts Python para garantir a sincronização correta na comunicação MQTT -> Kafka -> MongoDB, evitando perdas ou duplicação de mensagens.
- **Curva de aprendizado no Docker e orquestração:** especialmente na configuração das redes internas e volumes persistentes dos containers.

3.4 Validação da Solução

Apesar dos desafios encontrados, a solução proposta foi validada com sucesso. O sistema demonstrou ser capaz de:

- Coletar dados de forma contínua e confiável.
- Transmitir os dados de maneira eficiente via protocolo MQTT.
- Processar e rotear os dados utilizando Apache Kafka de forma escalável e resiliente.
- Persistir os dados no MongoDB, permitindo acesso, consultas e análises posteriores.

A arquitetura implementada se mostrou eficaz, modular e escalável, com potencial de expansão para múltiplos sensores, outros tipos de variáveis ambientais e integração com sistemas de alerta ou automação.

3.5 Pendências do Projeto

Identificamos uma limitação no consumo dos dados via MQTT. As mensagens estão chegando em lote, e como a configuração do sensor envia uma leitura a cada 10 segundos, o sistema demora para apresentar atualizações. Por conta do tempo disponível, não foi possível ajustar essa questão na etapa atual do projeto.

3.6 Discussão dos Resultados

A partir dos resultados obtidos, observa-se que o sistema desenvolvido atende aos objetivos propostos, permitindo o monitoramento ambiental contínuo e remoto. O comportamento das variáveis monitoradas refletiu adequadamente as condições do ambiente, incluindo variações induzidas por fontes de calor ou pelo controle dos aparelhos de ar-condicionado.

Além disso, a utilização da computação em nuvem, combinada com o ecossistema de containers via Docker e a visualização por meio da aplicação Flask, garantiu que a solução fosse robusta, escalável e facilmente replicável em outros ambientes.

CONCLUSÃO

4.1 Considerações Finais

Este trabalho desenvolveu uma solução de monitoramento ambiental baseada em *IoT* e computação em nuvem, utilizando MQTT, Apache Kafka, MongoDB e uma aplicação web em Flask. A arquitetura permite a coleta, transmissão, processamento e visualização dos dados de temperatura e umidade de forma estruturada e escalável.

O sistema foi capaz de realizar a coleta e o armazenamento dos dados de forma eficiente, com o fluxo completo MQTT → Kafka → MongoDB funcionando adequadamente. A visualização dos dados por meio do painel web validou a integração dos serviços e reforçou a utilidade da solução para monitoramento remoto.

4.2 Contribuições

As principais entregas deste projeto foram:

- Implementação de uma arquitetura modular e escalável, integrando dispositivos físicos com serviços na nuvem.
- Construção de um pipeline de dados robusto, empregando tecnologias amplamente utilizadas no mercado.
- Desenvolvimento de uma interface web para acompanhamento dos dados em tempo real.
- Criação de um ambiente de deployment via Docker Compose, simplificando a replicação da solução.

4.3 Desafios e Limitações

Os principais desafios enfrentados envolveram:

- Complexidade na configuração do Kafka e do Zookeeper.
- Instabilidades na conexão Wi-Fi do ESP32, afetando a transmissão de dados.

- Problemas na comunicação entre MongoDB e o frontend em Flask, devido a dificuldades na configuração das redes e portas do Docker.
- Atrasos no recebimento dos dados via MQTT, que acumulava mensagens e as entregava em lote.
- Necessidade de aprimorar o domínio sobre configuração de redes no Docker e técnicas de debug.

4.4 Trabalhos Futuros

Como aprimoramento futuro, sugere-se:

- Resolver os gargalos na comunicação MQTT e otimizar o pipeline de dados.
- Aprimorar a configuração das redes no Docker e o monitoramento dos serviços.
- Expandir o sistema para múltiplos sensores e outras variáveis, como luminosidade ou qualidade do ar.
- Adicionar alertas em tempo real e dashboards mais interativos.
- Integrar o sistema com nuvens públicas, como AWS, Azure ou Google Cloud.
- Incluir análises preditivas utilizando modelos de aprendizado de máquina.

4.5 Encerramento

A solução desenvolvida cumpriu os objetivos propostos, demonstrando ser viável, escalável e aplicável em cenários acadêmicos e industriais. A integração entre hardware e serviços de nuvem se mostrou eficaz, oferecendo uma plataforma robusta para monitoramento ambiental remoto.

4.6 Referências

- Github: <<https://github.com/ProfJCE-Disciplinas/cloud-project-ssc0158-2025-grupo-4>>
- Página Web: <<http://andromeda.lasdpc.icmc.usp.br:7154/>>
- Formulário: <<https://forms.gle/gdzJCwgfUJoAdQFr8>>